Messaging Systems Design

Interview Preparation Guide · SDE → Principal

Queues · Event Streaming · Pub/Sub · Distributed Messaging

SDE (L3–L4)	Senior SDE (L5)	Staff Engineer (L6)	Principal Engineer (L7+)

Level Expectations at a Glance

Topic	SDE	Senior	Staff	Principal
Core Concepts	Define queues/ topics	Compare systems	Design messaging infra	Org-wide strategy
Delivery Guarantees	At-least-once basics	Exactly-once tradeoffs	End-to-end guarantees	Platform SLAs
Ordering	FIFO basics	Partition ordering	Global ordering design	Novel ordering models
Scale & Throughput	Basic sizing	Partition tuning	Fleet-level capacity	Cost + efficiency models
Failure Handling	Retry logic	DLQ patterns	Chaos resilience	Platform fault domains
Observability	Lag monitoring	P99 latency tracking	Anomaly detection	Infra-wide tracing
Schema & Contracts	Awareness	Versioning basics	Schema registry design	Org data contracts

What interviewers expect: You understand what a message queue is, can use messaging systems in code, and reason about basic delivery guarantees and tradeoffs.

1. Core Concepts

- What is a message queue? Producer → Queue → Consumer model
- Topics vs Queues — broadcast (pub/sub) vs point-to-point
- Message broker vs event streaming — RabbitMQ vs Kafka
- Synchronous vs asynchronous communication — when to use each
- Why use messaging? Decoupling, buffering, async processing, fan-out

💡 *Be able to sketch: Producer → Broker → Consumer with ACK flow*

2. Delivery Guarantees

- At-most-once: fire and forget, possible message loss
- At-least-once: retry on failure, possible duplicates
- Exactly-once: no loss, no duplicates — hardest to achieve

💡 *At-least-once is most common in practice. Design consumers to be idempotent.*

3. Basic Messaging Patterns

Pattern	Description	Example Use Case
Point-to-Point	One producer, one consumer per message	Order processing
Pub/Sub	One producer, many consumers receive all messages	Notifications, event broadcast
Request/Reply	Producer waits for consumer response (async RPC)	API over messaging
Fan-out	Message copied to multiple queues/ consumers	Email + SMS + push notification
Competing Consumers	Multiple consumers share a queue (load balancing)	Worker pools, job queues

4. Message Acknowledgement

- ACK (Acknowledge) — consumer confirms successful processing
- NACK (Negative Acknowledge) — consumer signals failure, message re-queued
- Visibility timeout — message hidden from others while being processed
- Dead Letter Queue (DLQ) — where messages go after max retry attempts

💡 *Always design consumers to ACK only after successful processing, not on receipt.*

5. Key Technologies Overview

- RabbitMQ — traditional message broker, AMQP protocol, push-based
- Apache Kafka — distributed event log, pull-based, high throughput
- AWS SQS — managed queue, at-least-once, visibility timeout model
- AWS SNS — managed pub/sub, fan-out to SQS, Lambda, HTTP
- Google Pub/Sub — managed pub/sub, global, at-least-once
- Redis Streams — lightweight streaming with consumer groups

6. Basic Ordering

- FIFO queues — strict ordering, lower throughput
- Standard queues — best-effort ordering, higher throughput
- When ordering matters: financial transactions, state machine transitions
- When ordering doesn't matter: sending emails, analytics events

7. Interview Questions at This Level

- What is the difference between a queue and a pub/sub topic?
- What does "at-least-once delivery" mean and how do you handle duplicates?
- How would you use a message queue to decouple an order service from email notifications?
- What is a Dead Letter Queue and when would you use it?

What interviewers expect: You can architect messaging for production systems, choose the right technology, reason about ordering/consistency tradeoffs, and handle failure scenarios.

1. Kafka Deep Dive

- Topics → Partitions → Offsets — the core data model
- Consumer groups — each group gets all messages; consumers in group share partitions
- Partition assignment strategies: range, round-robin, sticky
- Replication factor and ISR (In-Sync Replicas) — durability guarantees
- Leader election on partition failure — automatic failover
- Log compaction — retain only latest value per key (useful for change data capture)
- Retention policy: time-based (7 days) vs size-based

💡 *Kafka is an append-only distributed log, not a traditional queue. Consumers pull at their own pace.*

2. Exactly-Once Semantics

- Idempotent producers — assign sequence numbers to detect/reject duplicates at broker
- Transactional producers — atomic write across multiple partitions/topics
- Kafka Transactions: begin → produce → commit (or abort)
- Consumer-side exactly-once: commit offset only after successful DB write (in same transaction)
- Exactly-once is expensive — evaluate if at-least-once + idempotent consumers suffice

💡 *True exactly-once requires coordination across producer, broker, AND consumer.*

3. Ordering Guarantees

- Partition-level ordering: messages in a partition are strictly ordered
- Key-based routing: same key always routes to same partition → ordered per key
- Global ordering: single partition (bottleneck) or external sequencer
- Out-of-order handling: event-time processing with watermarks (Flink, Spark)
- Sequence gaps: detect missing sequence numbers for ordered streams

4. Consumer Lag and Backpressure

- Consumer lag: how far behind consumers are from the latest offset
- Lag spike causes: consumer slowdown, processing errors, traffic burst
- Backpressure: slow consumers should not crash; pull model handles this naturally in Kafka
- Auto-scaling consumers based on lag metric (KEDA for Kubernetes)
- Partition count ≥ max consumer count (parallelism ceiling)

💡 *Monitor consumer lag per consumer group and alert when lag exceeds threshold*

5. Dead Letter Queue (DLQ) Design

- Retry policy: exponential backoff (1s → 2s → 4s → 8s → DLQ)
- Max retry count: 3–5 attempts before routing to DLQ
- DLQ consumer: separate service that alerts, inspects, replays, or discards
- Poison pill detection: messages that always fail (bad schema, business logic error)
- DLQ replay: reprocess fixed messages back to main queue

6. Push vs Pull Consumers

Dimension	Push (RabbitMQ)	Pull (Kafka, SQS)

Backpressure	Hard — broker overwhelms slow consumers	Natural — consumer fetches when ready
Latency	Lower (message pushed immediately)	Slightly higher (polling interval)
Throughput	Lower (per-message overhead)	Higher (batch fetch)
Complexity	Simpler consumer code	Consumer manages offsets
Use case	Low-volume, latency-sensitive	High-throughput, replay needed

7. Message Schema and Versioning

- Schema formats: JSON (flexible), Avro (compact, schema-enforced), Protobuf (typed, fast)
- Schema Registry (Confluent): centralized schema storage, compatibility enforcement
- Compatibility modes: BACKWARD (new readers read old messages), FORWARD, FULL
- Versioning in JSON: include a "version" field or use envelope pattern
- Breaking vs non-breaking schema changes — adding optional fields is safe

8. Messaging vs Direct API Calls

Criteria	Messaging	Direct API (REST/gRPC)
Coupling	Loose — producer unaware of consumers	Tight — must know endpoint
Availability	Broker absorbs spikes	Caller blocked if service down
Ordering	Configurable	Implicit (synchronous)
Debugging	Harder — async flow	Easier — direct call tracing
Latency	Higher (async)	Lower (synchronous)

9. Interview Questions at This Level

- Design a notification system that sends email, SMS, and push for 100M users
- How does Kafka guarantee message ordering within a topic?
- How do you handle duplicate message processing in a payment system?
- Design an event-driven order processing pipeline with retries and DLQ
- When would you choose RabbitMQ over Kafka?

What interviewers expect: You design messaging platforms, define cross-service contracts, reason about consistency under failure, and drive technical standards across teams.

1. Event-Driven Architecture (EDA)

- Event sourcing: store state changes as immutable events, derive state by replaying
- CQRS (Command Query Responsibility Segregation): separate write model (commands) from read model (queries)
- Choreography vs Orchestration:
  - Choreography: services react to events independently (loose coupling, hard to trace)
  - Orchestration: central coordinator drives workflow (easier to trace, tight coupling)
- Saga pattern: distributed transactions via compensating events
- Event replay: rebuild projections or recover state from event log

💡 *At staff level, articulate when EDA creates complexity vs value. Not every system needs it.*

2. Kafka Architecture at Scale

- Partition strategy: key selection determines ordering, hotspot risk, and parallelism
- Partition count tradeoffs: more partitions = more parallelism but more overhead (ZK, leader election)
- Tiered storage (Kafka 3.x): offload old segments to object storage (S3) for infinite retention
- KRaft (Kafka without ZooKeeper): simplified ops, faster leader election
- Kafka Streams: stateful stream processing directly in Kafka (joins, aggregations, windowing)
- Kafka Connect: standardized source/sink connectors (DB, S3, Elasticsearch)

3. Exactly-Once End-to-End

- Producer idempotence + transactions (broker-side)
- Outbox pattern: write event to DB in same transaction as business data, then relay to Kafka
- Transactional outbox vs dual-write: outbox is the only safe approach for DB + message atomicity
- Inbox pattern (idempotent consumer): deduplicate consumed messages using processed-ID store
- Offset management: store offsets in DB alongside processed results for atomic commit

4. Message Ordering at Scale

- Partition key design: userID for per-user ordering, orderID for per-order ordering
- Hot partition problem: popular keys route all traffic to one partition
- Mitigation: key salting (userID:0 to userID:N), separate hot-key topic
- Global ordering: single-partition topic (throughput bottleneck), use only when necessary
- Causally consistent ordering: vector clocks or Lamport timestamps for causal relationships

5. Messaging in Microservices

- Service mesh + messaging: async events for state changes, sync calls for queries
- Event contracts: define event schema as API contract between teams
- Event versioning governance: deprecation policy, sunset timelines
- Avoiding event spaghetti: bounded contexts, event ownership model
- Event catalog: centralized registry of events, schemas, owners, consumers

6. Failure Modes and Resilience

Failure	Impact	Mitigation
Broker failure	Partition unavailable	Replication + ISR + auto leader election

Consumer crash	Messages unprocessed (at-least-once)	ACK after processing, offset commit
Producer failure	Message loss or duplicate	Idempotent producer + retry with backoff
Network partition	Split-brain, stale consumers	ISR min.insync.replicas, consumer rebalance
Schema mismatch	Deserialization failure → DLQ	Schema registry + compatibility checks
Consumer lag spike	Delayed processing	Autoscale consumers, alert on lag
Poison pill message	Consumer crash loop	DLQ after max retries

7. Stream Processing

- Windowing: tumbling (non-overlapping), sliding (overlapping), session (gap-based)
- Watermarks: handle late-arriving events; advance processing time despite delays
- Stateful processing: joins, aggregations stored in state stores (RocksDB in Kafka Streams)
- Exactly-once processing: checkpointing in Flink, transactional sinks in Kafka Streams
- Lambda vs Kappa architecture: Lambda = batch + stream; Kappa = stream only (simpler)

8. Observability

- Producer metrics: send rate, error rate, retry rate, batch size
- Broker metrics: partition leader distribution, ISR count, log size, request latency
- Consumer metrics: lag per partition, fetch rate, commit rate, rebalance frequency
- End-to-end latency: from event creation timestamp to consumer processing timestamp
- Alerting: lag > threshold, ISR count < replication factor, DLQ message count > 0
- Distributed tracing: propagate correlation IDs through message headers

9. Interview Questions at This Level

- Design a messaging platform that 30 microservices can use with guaranteed delivery
- How do you implement the Saga pattern for a distributed checkout flow?
- Design an event sourcing system for a banking ledger
- How do you handle schema evolution across teams producing and consuming the same topic?
- You detect a consumer group with 2 million messages of lag. Walk me through diagnosis.

What interviewers expect: You define messaging infrastructure strategy at org scale, evaluate build vs buy, shape data contracts across hundreds of services, and contribute to industry thinking.

1. Messaging as a Platform

- Self-service messaging platform: provisioning, quota management, schema governance
- Multi-tenancy: namespace isolation, per-team quotas, noisy neighbor prevention
- Platform SLA definition: latency, throughput, durability, availability guarantees
- Cost allocation: chargeback model per team based on message volume and storage
- Abstraction layer: platform API that hides Kafka internals from application developers

2. Data Mesh and Event Mesh

- Data mesh: treat data (events) as a product; domain teams own their event streams
- Event mesh: network of brokers enabling routing across regions, clouds, protocols
- Data contracts: formal schema + SLA agreement between producer and consumer teams
- Federated governance: centralized standards (schema compatibility) + decentralized ownership
- Event-driven data platform: Kafka as the backbone for real-time data products

3. Global Messaging Architecture

- Multi-region Kafka: MirrorMaker 2.0 for cross-region replication
- Active-active vs active-passive multi-region: tradeoffs in consistency and complexity
- Geo-routing: route producer writes to nearest region, replicate asynchronously
- Conflict resolution for multi-region writes: last-write-wins, CRDTs, application-level merge
- Regulatory compliance: data residency, GDPR — prevent EU events flowing to US clusters

4. Advanced Delivery and Ordering

- Total order broadcast: equivalent to consensus (Paxos/Raft) — expensive at scale
- Causal consistency: deliver events in causal order without global coordination
- Vector clocks in messaging: track causality without central sequencer
- Hybrid logical clocks (HLC): combine physical and logical time for causal ordering
- Deterministic message routing: consistent hashing for stable partition assignment

5. Messaging Economics and Efficiency

- Cost drivers: storage (retention), network (replication, cross-AZ), compute (consumers)
- Compression strategy: lz4 for throughput, zstd for compression ratio, snappy for balance
- Batching optimization: linger.ms + batch.size tuning for throughput vs latency
- Tiered storage ROI: move cold data to S3 — 10-100x storage cost reduction
- Autoscaling cost model: dynamic consumer scaling to match lag, turn off idle consumers

6. Emerging Patterns and Technologies

- Apache Pulsar: multi-tenancy native, tiered storage built-in, geo-replication, separate compute/storage
- NATS JetStream: lightweight, high performance, cloud-native messaging
- Redpanda: Kafka-compatible, no ZooKeeper, C++ implementation (lower latency)
- WarpStream: Kafka-compatible, runs entirely on object storage (S3), zero disk cost
- CloudEvents spec: standard event envelope format for portability across brokers
- AsyncAPI: OpenAPI equivalent for event-driven APIs — documenting messaging contracts

7. Build vs Buy Analysis

Option	Best For	Key Tradeoff
Kafka OSS	High throughput, event sourcing, replay	Ops complexity, ZK/KRaft management
Confluent Cloud	Managed Kafka, enterprise features	Cost, vendor dependency
AWS MSK	AWS-native Kafka	Less control, AWS lock-in
AWS SQS/SNS	Serverless, simple queues	No replay, limited ordering
Google Pub/Sub	GCP native, global, serverless	No consumer groups concept
Apache Pulsar	Multi-tenancy, geo-replication	Smaller ecosystem vs Kafka
Redpanda	Low latency, Kafka-compatible	Newer, smaller community

8. Organizational Standards

- Event naming conventions: domain.entity.event (e.g., payments.order.completed)
- Mandatory envelope fields: eventId, eventType, source, timestamp, correlationId, schemaVersion
- Retention policy standards: default 7 days; compliance topics 7 years
- Schema review process: schema changes require producer team + consumer team sign-off
- Runbook requirements: every topic must have documented DLQ and consumer runbook

9. Interview Questions at This Level

- Design the messaging infrastructure for a company processing 10 billion events per day
- How do you migrate 200 services from RabbitMQ to Kafka with zero downtime?
- Design a real-time fraud detection pipeline that processes 1M transactions/second
- How do you enforce data contracts across 50 engineering teams in a data mesh architecture?
- Kafka vs Pulsar vs Redpanda — when would you choose each, and what drives that decision?

Quick Reference: Technology Comparison

System	Model	Ordering	Replay	Best For
Apache Kafka	Log/Stream	Per partition	Yes (retention)	High throughput, event sourcing
RabbitMQ	Queue	Per queue	No (after ACK)	Complex routing, low-latency
AWS SQS	Queue	FIFO optional	No	Serverless, simple decoupling
AWS SNS	Pub/Sub	No guarantee	No	Fan-out, notifications
Google Pub/Sub	Pub/Sub	Per key	Within retention	GCP native, global
Apache Pulsar	Log + Queue	Per partition	Yes	Multi-tenant, geo-replication
Redis Streams	Log	Yes (ID-based)	Yes (in memory)	Lightweight, low-latency

Top 5 System Design Scenarios

Scenario	SDE	Senior	Staff	Principal
Notification System	SNS + SQS fan-out	Retry + DLQ strategy	Rate limiting + batching	Global delivery + compliance
Order Processing	Queue per step	Saga + compensation	Event sourcing + CQRS	Transactional consistency SLA
Activity Feed	Pub/sub basics	Fan-out at scale	Hybrid push/pull model	Data mesh + real-time ML
Fraud Detection	Event triggers	Stream processing	CEP + stateful windows	1M TPS + sub-100ms SLA
Audit Log	Write to queue	Immutable log design	Event sourcing + replay	Compliance + geo-residency

Good luck with your interview preparation! 🚀